## Lab 04: Creational Design Patterns

### Learning Objectives

By the end of this lab, you should be able to:

- Understand the concept of Creational Design Patterns.
- Explain different approaches to object creation.
- Implement Factory Method, Abstract Factory, Builder, Prototype, and Singleton patterns
- Identify the advantages and limitations of each pattern.

### Creational Design Patterns

Creational Design Patterns are design patterns that deal with **how objects are created**. They provide alternatives to directly creating objects using the new keyword and help make object creation more flexible, reusable, and maintainable.

Instead of writing:

```
Delivery delivery = new BikeDelivery();
```

a creational pattern can control or encapsulate the object-creation process.

### Common Creational Patterns

This lab covers:

1. Factory Method
2. Abstract Factory
3. Builder
4. Prototype
5. Singleton

### 1. Factory Method Pattern

The Factory Method Pattern provides a method for creating objects without requiring the client to know the exact class being instantiated. The client asks the factory for an object, and the factory determines which concrete object should be created.

### When to Use

Use Factory Method when:

- Multiple types of objects can be created.
- The required object type is determined at runtime.
- You want to separate object creation from the code that uses the object.

### Example: Food Delivery System

Suppose a food delivery system supports bike, car, and drone delivery.

### Step 1: Create the Product Interface

```
interface Delivery {
    void deliver();
}
```

**Step 2: Create Concrete Products**

```
class BikeDelivery implements Delivery {
   public void deliver() {
      System.out.println("Order delivered by bike.");
   }
}
class CarDelivery implements Delivery {
   public void deliver() {
      System.out.println("Order delivered by car.");
   }
}
class DroneDelivery implements Delivery {
   public void deliver() {
      System.out.println("Order delivered by drone.");
   }
}
```

**Step 3: Create Factory**

```
class DeliveryFactory {
   public static Delivery createDelivery(String type) {
      if (type.equalsIgnoreCase("bike")) {
         return new BikeDelivery();
      }
      else if (type.equalsIgnoreCase("car")) {
         return new CarDelivery();
      }
      else if (type.equalsIgnoreCase("drone")) {
         return new DroneDelivery();
      }
      throw new IllegalArgumentException(
            "Unknown delivery type");
   }
}
```

**Step 4: Use Factory**

```
public class Main {
   public static void main(String[] args) {
      Delivery delivery = DeliveryFactory.createDelivery("drone");
      delivery.deliver();
   }
}
```

**2. Abstract Factory Pattern**

The Abstract Factory Pattern provides an interface for creating **families of related objects** without specifying their concrete classes. Unlike Factory Method, which generally focuses on creating a particular type of product, Abstract Factory creates a group of related products that are designed to work together.

Use Abstract Factory when:

- You need multiple related objects.
- The objects should be compatible with each other.
- You want to switch between complete families of objects.

**Example**

A food delivery system has different types of order. Each order type uses a compatible payment and notification service.

For example:

**Online Order**

- Credit Card Payment
- Email Notification

**Local Order**

- Cash Payment
- SMS Notification

**Step 1: Product Interfaces**

```
interface Payment {
   void pay(); }
interface Notification {
   void send(); }
```

**Step 2: Concrete Products**

```
class CreditCardPayment implements Payment {
   public void pay() {
      System.out.println("Payment made using credit card.");
   } }
class CashPayment implements Payment {
   public void pay() {
      System.out.println("Payment made using cash.");
   } }
class EmailNotification implements Notification {
   public void send() {
      System.out.println("Notification sent through email.");
   } }
class SMSNotification implements Notification {
   public void send() {
         System.out.println("Notification sent through SMS.");
} }
```

**Step 3: Abstract Factory**

```
interface DeliveryServiceFactory {
    Payment createPayment();
    Notification createNotification();
}
```

**Step 4: Concrete Factories**

```
class OnlineOrderFactory implements DeliveryServiceFactory {
    public Payment createPayment() {
        return new CreditCardPayment();
    }
    public Notification createNotification() {
        return new EmailNotification();
    }
}
class LocalOrderFactory implements DeliveryServiceFactory {
    public Payment createPayment() {
        return new CashPayment();
    }
    public Notification createNotification() {
        return new SMSNotification();
    }
}
```

**Step 5: Use Abstract Factory**

```
public class Main {
    public static void main(String[] args) {
        DeliveryServiceFactory factory =
                new OnlineOrderFactory();
        Payment payment = factory.createPayment();
        Notification notification =
                factory.createNotification();
        payment.pay();
        notification.send();
    }
}
```

**3. Builder Pattern**

The Builder Pattern separates the construction of a complex object from its representation and allows the object to be created **step by step**. It is particularly useful when a class has many optional attributes.

**Problem Without Builder**

Imagine an order has:

- Customer Name
- Restaurant
- Address
- Payment Method
- Instructions
- Discount
- Delivery Time

A constructor containing all these parameters can become difficult to read and use.

Use Builder when:

- An object has many attributes.
- Many attributes are optional.
- Object creation involves multiple steps.
- You want readable object construction.

**Example**

```
class FoodOrder {
    private String customerName;
    private String restaurant;
    private String address;
    private String paymentMethod;
    private String instructions;
    private FoodOrder(Builder builder) {
        this.customerName = builder.customerName;
        this.restaurant = builder.restaurant;
        this.address = builder.address;
        this.paymentMethod = builder.paymentMethod;
        this.instructions = builder.instructions;
    }
    public void displayOrder() {
        System.out.println("Customer: " + customerName);
        System.out.println("Restaurant: " + restaurant);
        System.out.println("Address: " + address);
        System.out.println("Payment: " + paymentMethod);
        System.out.println("Instructions: " + instructions);
    }
    static class Builder {
        private String customerName;
        private String restaurant;
        private String address;
        private String paymentMethod;
```

```
    private String instructions;
    public Builder(String customerName,
                   String restaurant) {
      this.customerName = customerName;
      this.restaurant = restaurant;
    }
    public Builder setAddress(String address) {
      this.address = address;
      return this;
    }
    public Builder setPaymentMethod(String paymentMethod) {
      this.paymentMethod = paymentMethod;
      return this;
    }
    public Builder setInstructions(String instructions) {
      this.instructions = instructions;
      return this;
    }
    public FoodOrder build() {
      return new FoodOrder(this);
    }
  }
}
```

**Creating an Object**

```
public class Main {
  public static void main(String[] args) {
    FoodOrder order =
        new FoodOrder.Builder(
            "Ali", "Pizza House")
        .setAddress("Islamabad")
        .setPaymentMethod("Credit Card")
        .setInstructions("Deliver after 7 PM")
        .build();
    order.displayOrder();
  }}
```

The Builder allows us to write:

- setAddress(...)
- setPaymentMethod(...)
- setInstructions(...)
- build();

instead of passing many parameters to a constructor.

**4. Prototype Pattern**

The Prototype Pattern creates a new object by **copying or cloning an existing object**, known as the prototype. Instead of constructing a completely new object, we create a copy of an existing one and modify it if necessary.

Use Prototype when:

- Objects have similar configurations.
- Creating an object from scratch is expensive or complicated.
- You need many objects with similar initial states.

**Example**

Suppose a restaurant already has a menu item and wants to create similar menu items.

```
class MenuItem implements Cloneable {
  private String name;
  private double price;
  public MenuItem(String name, double price) {
    this.name = name;
    this.price = price;
  }
  public void setPrice(double price) {
    this.price = price;
  }
  public void display() {
    System.out.println(name + " - $" + price);
  }
  @Override
  public MenuItem clone() {
    try {
      return (MenuItem) super.clone();
    }
    catch (CloneNotSupportedException e) {
      throw new RuntimeException(e);
    }}}
```

**Using Prototype**

```
public class Main {
  public static void main(String[] args) {
    MenuItem original = new MenuItem("Chicken Burger", 8.0);
    MenuItem copy = original.clone();
    copy.setPrice(7.0);
    original.display();
    copy.display();
  }
}
```

**5. Singleton Pattern**

The Singleton Pattern ensures that a class has **only one instance** and provides a global access point to that instance.

Singleton can be used when:

- Exactly one instance of a class is required.
- The same shared object must be accessed throughout the application.

Examples can include certain configuration or resource-management objects.

**Example**

```
class AppConfig {
   private static AppConfig instance;
   private AppConfig() {
   }
   public static AppConfig getInstance() {
      if (instance == null) {
         instance = new AppConfig();
      }
      return instance;
   }
   public void displayMessage() {
      System.out.println(
            "Application configuration loaded.");
   }
}
```

**Using Singleton**

```
public class Main {
   public static void main(String[] args) {
      AppConfig config1 = AppConfig.getInstance();
      AppConfig config2 = AppConfig.getInstance();
      System.out.println(config1 == config2);
   }
}
```

Both variables refer to the **same object**.

**Singleton Pitfalls**

**1. Global State**

A Singleton behaves like globally accessible state, which can make it difficult to understand which part of the application is modifying its data.

**2. Difficult Testing**

Classes that directly depend on a Singleton can be harder to test because replacing the Singleton with a test object is not always straightforward.

**3. Hidden Dependencies**

A class may access the Singleton internally without explicitly showing that dependency in its constructor or method parameters.

**4. Multithreading Issues**

A simple Singleton implementation may create multiple instances if several threads try to create the instance simultaneously.

**5. Overuse**

Not every shared object needs to be a Singleton. Using it unnecessarily can make the design less flexible.